TEC | Tecnológico de Costa Rica

Proyecto #2

Compiladores e intérpretes

Análisis Semántico y Generación de Código Intermedio

Estudiantes:

Tayler Wynta Rodríguez | 2024143103
Maikel Flores Navarro | 2024148346

Profesor:

Ing. Allan Rodríguez Dávila

II Semestre 2026

Contenidos

Manual de usuario 3
Instrucciones de compilación, ejecución y uso. 3
Pruebas de funcionalidad 4
Caso válido 5
Caso inválido 9
Descripción del problema. 12
Diseño del programa: decisiones de diseño, algoritmos usados. 12
Análisis de resultados: objetivos alcanzados, objetivos no alcanzados, y razones por las cuales no se alcanzaron los objetivos (en caso de haberlos). 16
Justificación de toma de decisiones: forma en que se realizó la validación semántica y sus estructuras. .. 17
Bitácora 19

Manual de usuario

Instrucciones de compilación, ejecución y uso.

Todos los comandos se ejecutan desde la carpeta Programa/ del repositorio, en una terminal de Windows (PowerShell). Se necesita tener instalado un JDK.

1. Generar Lexer.java desde la especificación léxica:

```
java -jar lib\jflex-full-1.9.1.jar --encoding UTF-8 -d src especificaciones\Lexer.flex
```

2. Generar Parser.java y sym.java desde la gramática:

```
java -cp lib\java-cup-11b-20151001.jar java_cup.Main -destdir src -parser Parser -symbols
sym especificaciones\Parser.cup
```

3. Compilar:

```
New-Item -ItemType Directory -Force out
javac -encoding UTF-8 -cp lib\java-cup-runtime-11b-20151001.jar -d out src\Main.java
src\Lexer.java src\Parser.java src\sym.java src\TablaSimbolos.java
src\ResultadoExpresion.java src\GeneradorCodigoIntermedio.java
```

Ejecución

Asegurarse de que exista la carpeta salida/ (si no existe, crearla; el programa no la crea solo). Desde la carpeta Programa/:

```
New-Item -ItemType Directory -Force salida
java "-Dfile.encoding=UTF-8" -cp "out;lib\java-cup-runtime-11b-20151001.jar" Main
```

El -Dfile.encoding=UTF-8 es necesario porque el lenguaje usa símbolos especiales (ⱶ, λ, θ, Σ, є, ɜ, ∫, ɳ, ¿, », etc.) y sin forzar la codificación se pueden leer o imprimir mal según la configuración de la consola.

Uso

El programa analiza el archivo pruebas/Prueba.txt. Para probar otro programa basta con reemplazar el contenido de ese archivo y volver a ejecutar. En una sola corrida se realizan las cuatro fases (léxica, sintáctica, semántica y generación de código intermedio) y se generan los siguientes archivos:

- Imprime en consola el resultado de cada fase (léxico, sintáctico y semántico) y, al final, si el archivo pertenece o no a la gramática. Solo se acepta si no hubo errores en ninguna de las tres fases; si no, indica en qué archivo ver los errores.
- salida/tokens.txt: cada token encontrado, su lexema y la línea donde apareció.

- salida/errores_lexicos.txt: errores léxicos encontrados (si hubo).
- salida/errores_sintacticos.txt: errores sintácticos encontrados, con línea y columna (si hubo).
- salida/errores_semanticos.txt: errores semánticos encontrados, con la línea y una descripción del problema (si hubo).
- salida/tabla_simbolos.txt: la tabla de símbolos, separada por ámbito (Global, funciones, bloques, ciclos y condicionales).
- salida/codigo_intermedio.txt: el código intermedio de tres direcciones generado para el archivo fuente.

Si el archivo tiene errores, el programa no se detiene en el primero: sigue revisando el resto del archivo y reporta todos los errores léxicos, sintácticos y semánticos que encuentre en esa misma corrida.

Pruebas de funcionalidad

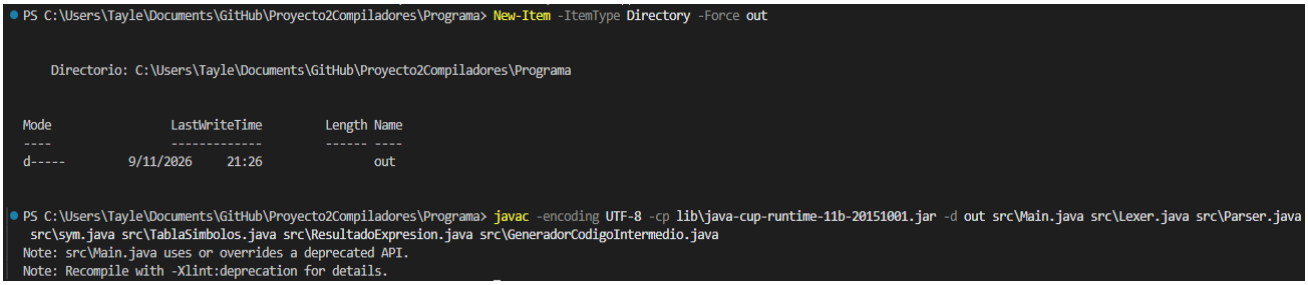
Ejecutando en consola el comando para crear el lexer, se genera correctamente sin errores (140 estados en el DFA minimizado).

```
PS C:\Users\Tayle\Documents\GitHub\Proyecto2Compiladores\Programa> java -jar lib\jflex-full-1.9.1.jar --encoding UTF-8 -d src especificaciones\Lexer.flex
Reading "especificaciones\Lexer.flex"
Constructing NFA : 371 states in NFA
Converting NFA to DFA :
.....
156 states before minimization, 140 states in minimized DFA
Old file "src\Lexer.java" saved as "src\Lexer.java~"
Writing code to "src\Lexer.java"
```

Ejecutando en consola el comando para crear el parser, se genera correctamente sin errores y con 0 conflictos (158 producciones, 281 estados LALR).

```
PS C:\Users\Tayle\Documents\GitHub\Proyecto2Compiladores\Programa> java -cp lib\java-cup-11b-20151001.jar java_cup.Main -destdir src -parser Parser -symbols sym especificaciones\Pars
er.cup
------- CUP v0.11b 20150930 (SVN rev 66) Parser Generation Summary -------
  0 errors and 0 warnings
  52 terminals, 76 non-terminals, and 158 productions declared,
  producing 281 unique parse states.
  0 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "Parser.java", and "sym.java".
---------------------------------------------------- (CUP v0.11b 20150930 (SVN rev 66))
```

Al compilar todas las clases juntas (Main, Lexer, Parser, sym, TablaSimbolos, ResultadoExpresion y GeneradorCodigoIntermedio) lo hace de forma compatible y sin errores.



Caso válido

En este caso se ejecutó un programa léxica, sintáctica y semánticamente correcto. Usa una función con parámetros y return, un arreglo inicializado, una llamada a función con argumentos, un ciclo while con un if anidado que hace break, y un write. El código fuente usado fue:

```
int sumar ϵ:int x, int y:ɜ ¿:
  val int r Ⱶ x + y»
  return r»
:?

void principalϵ::ɜ ¿:
  val int a Ⱶ 10»
  val int lista ʃ:3:ɳ Ⱶ ¿: 1, 2, 3 :?»
  val int total Ⱶ sumarϵ:a, 5:ɜ»
  while ϵ:ʃ:a:ɳ > ʃ:0:ɳ:ɜ ¿:
    a Ⱶ a - 1»
    if ϵ:ʃ:a:ɳ == ʃ:5:ɳ:ɜ ¿:
      break»
    :?
  :?
  write ϵ:total:ɜ»
:?
```

Resultado en consola:

Programa > pruebas > Prueba.txt

You, 3 days ago | authors (You and one other)

```
int sumar ϵ:int x, int y:ə ¿:
  val int r ⊢ x + y»
  return r»
:?

void principalϵ::ə ¿:
  val int a ⊢ 10»
  val int lista ʃ:3:l ⊢ ¿: 1, 2, 3 :?»
  val int total ⊢ sumarϵ:a, 5:ə»
  while ϵ:ʃ:a:l > ʃ:0:l:ə ¿:
    a ⊢ a - 1»
    if ϵ:ʃ:a:l == ʃ:5:l:ə ¿:
      break»
    :?
  :?
  write ϵ:total:ə»
:?
```

Problems Output Debug Console Terminal Ports powershell - Programa

```
PS C:\Users\Tayle\Documents\GitHub\Proyecto2Compiladores\Programa> java "-Dfile.encoding=UTF-8" -cp "out;lib\java-cup-runtime-11b-20151001.jar" Main
Analisis lexico finalizado con exito.
Analisis sintactico finalizado con exito.
Analisis semantico finalizado con exito.
Tabla de simbolos guardada en: salida/tabla_simbolos.txt
Codigo intermedio guardado en: salida/codigo_intermedio.txt

El programa pertenece a la gramatica.
```

Tokens generados:

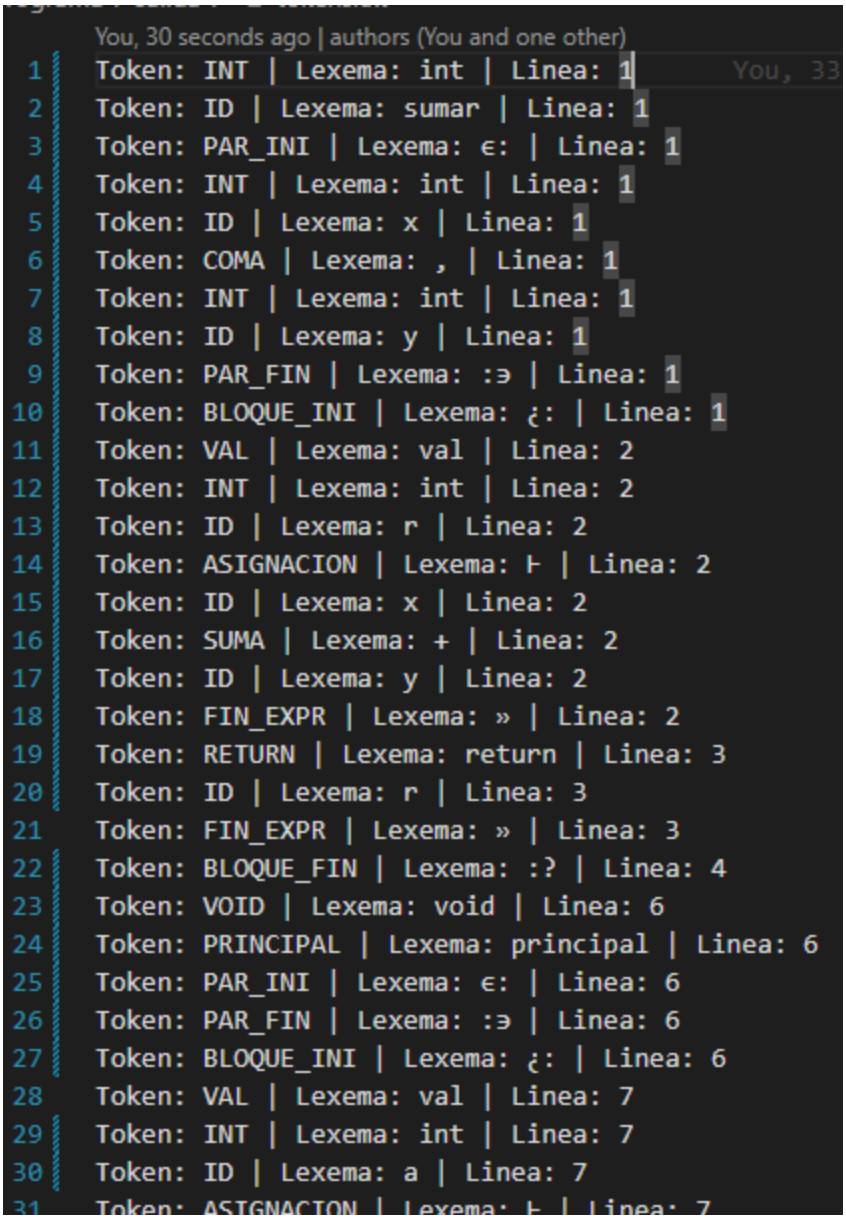


Tabla de símbolos generada:

```
=== TABLA DE SIMBOLOS: Global ===      Maikel Flores, last week
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
sumar                int          funcion         1

=== TABLA DE SIMBOLOS: Global / funcion sumar ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
x                    int          parametro       1
y                    int          parametro       1
r                    int          variable        2

=== TABLA DE SIMBOLOS: Global / principal ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
a                    int          variable        7
lista                int[]        arreglo         8
total                int          variable        9

=== TABLA DE SIMBOLOS: Global / principal / while 1 ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
(sin simbolos)

=== TABLA DE SIMBOLOS: Global / principal / while 1 / if 1 ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
(sin simbolos)
```

Código intermedio generado:

```
1   func sumar:
2   param_int x
3   param_int y
4   t0 = x + y
5   data_int r
6   r = t0
7   return r
8   endfunc sumar
9   func principal:
10  data_int a
11  t1 = 10
12  a = t1
13  data_int lista[3]
14  t2 = 1
15  lista[0] = t2
16  t3 = 2
17  lista[1] = t3
18  t4 = 3
19  lista[2] = t4
20  t5 = 5
21  param a
22  param t5
23  t6 = call sumar, 2
24  data_int total
25  total = t6
26  principal_while_0_inicio:
27  t7 = a > 0
28  if t7 goto principal_while_0_bloque
29  goto principal_while_0_fin
30  principal_while_0_bloque:
31  t8 = a - 1
```

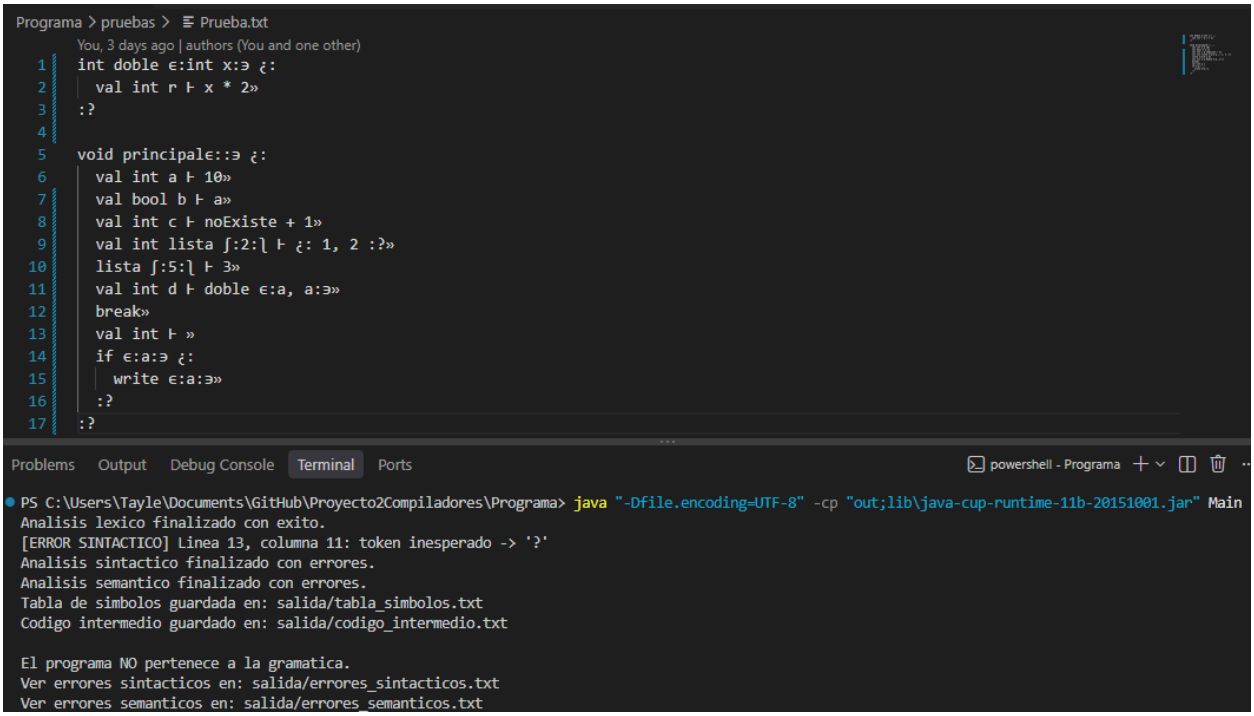
Caso inválido

Este es un caso de prueba con errores sintácticos y semánticos mezclados, para comprobar que el análisis no se detiene en el primer error y que cada tipo de error se reporta en su archivo. Contiene: una función sin return, una asignación con tipos incompatibles, una variable no declarada, un índice fuera de rango, una llamada con cantidad incorrecta de argumentos, un break fuera de un ciclo, una declaración mal formada (error sintáctico) y una condición que no es booleana. El código fuente usado fue:

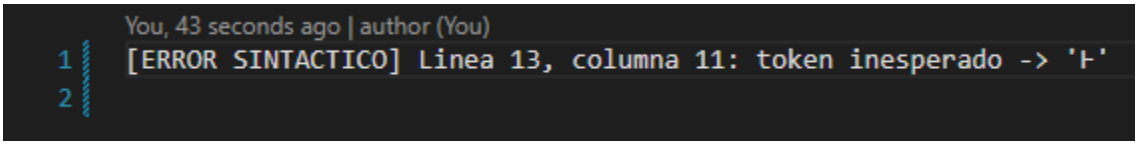
```
int doble є:int x:ɔ ¿:
  val int r Ⱶ x * 2»
:?
```

```
void principalϵ::ɜ ¿:
  val int a ⊦ 10»
  val bool b ⊦ a»
  val int c ⊦ noExiste + 1»
  val int lista ʃ:2:ʅ ⊦ ¿: 1, 2 :?»
  lista ʃ:5:ʅ ⊦ 3»
  val int d ⊦ doble ϵ:a, a:ɜ»
  break»
  val int ⊦ »
  if ϵ:a:ɜ ¿:
    write ϵ:a:ɜ»
  :?
:?
```

Resultado en consola:



Errores sintácticos encontrados:



Errores semánticos encontrados:

```
[ERROR SEMANTICO] Linea 1: la funcion 'doble' debe contener al menos un return.
[ERROR SEMANTICO] Linea 7: no se puede asignar un valor de tipo 'int' a la variable 'b' de tipo 'bool'.
[ERROR SEMANTICO] Linea 8: el identificador 'noExiste' no ha sido declarado o esta fuera de su ambito.
[ERROR SEMANTICO] Linea 10: indice 5 fuera de los limites del arreglo 'lista' de tamaño 2.
[ERROR SEMANTICO] Linea 11: la funcion 'doble' espera 1 argumento(s), pero recibio 2.
[ERROR SEMANTICO] Linea 12: break solo puede utilizarse dentro de un ciclo.
[ERROR SEMANTICO] Linea 14: la condicion de 'if' debe ser de tipo 'bool', pero se obtuvo 'int'.

```

Tabla de símbolos generada:

```
You, 3 minutes ago | authors (You and one other)
=== TABLA DE SIMBOLOS: Global ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
doble                int          funcion         1

=== TABLA DE SIMBOLOS: Global / funcion doble ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
x                    int          parametro       1
r                    int          variable        2

=== TABLA DE SIMBOLOS: Global / principal ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
a                    int          variable        6
b                    bool         variable        7
c                    int          variable        8
lista                int[]        arreglo         9
d                    int          variable        11

=== TABLA DE SIMBOLOS: Global / principal / if 1 ===
Nombre               Tipo         Categoria       Linea
------               ----         ---------       -----
(sin simbolos)
```

Descripción del problema.

Un grupo de desarrolladores quiere un lenguaje imperativo nuevo y liviano, pensado para configurar chips y para sistemas empotrados, donde no se necesita todo lo que trae un lenguaje grande sino algo simple y directo para hacer operaciones básicas. En el Proyecto I construimos las dos primeras fases del compilador: el analizador léxico (JFlex) y el analizador sintáctico (CUP), junto con la tabla de símbolos.

En este segundo proyecto había que continuar con las dos fases siguientes, tomando como base el mismo .flex y .cup. Primero el análisis semántico: ya no basta con que el código esté bien escrito según la gramática, también tiene que "tener sentido". El lenguaje es de tipado explícito y fuerte, así que hay que verificar que cada identificador esté declarado en un ámbito accesible, que los tipos de las operaciones, asignaciones, condiciones, argumentos y retornos sean compatibles, que los arreglos se usen bien, que break solo aparezca dentro de un ciclo, que las funciones con tipo tengan return, y que el programa tenga su función principal.

Después, la generación de código intermedio: para un programa correcto hay que escribir en un archivo su equivalente en código de tres direcciones, con el mismo sentido semántico que el código fuente. Es decir, descomponer las expresiones en temporales, y traducir los if, elif, else, while, for, break, llamadas a funciones, arreglos, read y write a instrucciones simples con etiquetas y saltos.

Igual que en el primer proyecto, todo tiene que trabajar sobre el mismo archivo fuente, decir si pertenece o no a la gramática, y manejar los errores léxicos, sintácticos y semánticos con recuperación en modo pánico, de forma que un solo error no tumbe el análisis y en una sola corrida se reporten todos los problemas con su línea.

Diseño del programa: decisiones de diseño, algoritmos usados.

Cómo está armado por dentro

El programa sigue con la misma idea del Proyecto I, pero ahora con más piezas. El Lexer lee el archivo y saca los tokens. El Parser toma esos tokens, arma la estructura gramatical y, en la misma pasada, hace el análisis semántico y genera el código intermedio. La TablaSimbolos guarda los ámbitos y lo que se declara en cada uno. ResultadoExpresion es el objeto que se pasan entre sí las reglas de las expresiones. El GeneradorCodigoIntermedio va acumulando las instrucciones de tres direcciones. Y Main.java coordina todo: corre una pasada léxica (tokens.txt y errores_lexicos.txt), después corre el parser sobre el mismo archivo y al final escribe los errores sintácticos, los semánticos, la tabla de símbolos y el código intermedio. El veredicto final toma en cuenta las tres fases: el archivo solo se acepta si no hubo errores léxicos, sintácticos ni semánticos, y si los hubo, indica en qué archivo de salida revisarlos.

Archivo	Responsabilidad
especificaciones/Lexer.flex	Especificación léxica (heredada del Proyecto I).
especificaciones/Parser.cup	Gramática, acciones semánticas y llamadas al generador de código.
src/Main.java	Punto de entrada; corre las fases y escribe los archivos de salida.
src/TablaSimbolos.java	Pila de ámbitos, símbolos, firmas de funciones y tamaños de arreglos.
src/ResultadoExpresion.java	Atributo sintetizado de las expresiones: tipo y lugar (temporal/variable).
src/GeneradorCodigoIntermedio.java	Temporales, etiquetas, contextos de control de flujo y escritura del archivo.

Traducción dirigida por la sintaxis: una sola pasada

No construimos un árbol sintáctico aparte para recorrerlo después. Usamos la técnica de traducción dirigida por la sintaxis: cada producción del .cup tiene acciones en Java que se ejecutan en el momento en que CUP la reduce. Como el parser es LALR(1) (de abajo hacia arriba), cuando se reduce una expresión ya se conocen los tipos de sus operandos, así que es el lugar natural para validar los tipos y generar el código de esa expresión. Es una gramática con atributos sintetizados: el tipo y el lugar de cada expresión suben desde las hojas (literales e identificadores) hasta la raíz.

Para las cosas que tienen que pasar antes de terminar una regla, como abrir el ámbito de una función antes de leer sus parámetros o emitir la etiqueta de inicio de un while antes de evaluar la condición, usamos acciones intermedias de CUP, que se ejecutan a la mitad de la producción.

El análisis semántico

Toda la validación vive en métodos del bloque parser code del .cup (validarOperacionAritmetica, validarOperacionRelacional, validarLlamadaFuncion, validarAccesoArreglo, validarReturn, validarBreak, etc.). Cada uno recibe los tipos ya calculados, revisa la regla correspondiente y, si algo no calza, agrega un mensaje con la línea a la lista erroresSemanticos. Nunca se tira una excepción por un error semántico: se registra y el parser sigue normal.

Cuando una expresión tiene un error, su resultado queda en null. Todas las validaciones de arriba revisan si les llega un null y, si es así, no reportan nada nuevo. Así se evita la cascada de errores: si noExiste + 1 falla porque la variable no existe, no aparece además un segundo error diciendo que no se puede asignar a c.

La tabla de símbolos

Se mantuvo la pila de ámbitos del Proyecto I, pero se le agregó información que el análisis semántico necesitaba:

- Cada símbolo tiene categoría (variable, parametro, funcion o arreglo), tipo y línea.
- Las funciones guardan su firma: la lista ordenada de tipos de sus parámetros.
- Los arreglos guardan su tamaño, para poder validar índices constantes.
- buscarSimbolo recorre la pila desde el ámbito actual hasta el global, que es lo que define el alcance léxico: lo de afuera se ve desde adentro, pero no al revés.
- Los ciclos y condicionales abren ámbitos con nombre (por ejemplo, while 1, for 2), lo que permite saber si un break está dentro de un ciclo y en qué función estamos (para validar el return).

La generación de código intermedio

El código que se genera es de tres direcciones: cada instrucción tiene a lo sumo un operador y tres operandos. Las instrucciones que usamos son:

Instrucción	Significado
func f: / endfunc f	Inicio y fin de una función.
param_int x	Declaración de un parámetro formal de la función.
data_int a / data_int v[3]	Declaración de una variable o de un arreglo.
t0 = 10	Carga de un literal en un temporal.
t1 = a + b	Operación binaria (aritmética, relacional o lógica).
t2 = Σ c	Negación lógica.
a = t1	Asignación.
t3 = v[i] / v[0] = t2	Lectura y escritura de un elemento de arreglo.
param t5 + t6 = call f, 2	Paso de argumentos y llamada a función (call sin asignar si es void).
return r	Retorno de una función.
L: / goto L	Etiqueta y salto incondicional.
if t goto L / ifFalse t goto L	Saltos condicionales.
read a / write t	Entrada y salida.

Temporales: cada valor intermedio se guarda en un temporal nuevo (t0, t1, …) que da nuevoTemporal(). Los literales también se cargan en un temporal antes de usarse, así cada operación trabaja solo con nombres.

Etiquetas legibles: en vez de etiquetas genéricas (L1, L2), las etiquetas llevan el nombre de la función, la estructura y un contador, por ejemplo principal_while_0_inicio o sumar_if_3_else_bloque. Esto hace que el código intermedio sea mucho más fácil de leer y depurar.

Control de flujo con pilas de contexto: como los if, while y for se pueden anidar, el generador guarda una pila de contextos para cada uno. Al entrar a una estructura se apila su contexto (con sus etiquetas) y al salir se desapila. Así un if dentro de un while sabe a qué etiquetas saltar sin confundirse con las del while.

if / elif / else: se evalúa la condición y se emite ifFalse cond goto siguiente. Cada elif emite un goto fin para el bloque anterior y su propia etiqueta de evaluación; el else hace lo mismo, y al final se coloca la etiqueta de fin.

for: el incremento aparece en el encabezado, pero tiene que ejecutarse después del cuerpo. Para eso, mientras se analiza el incremento, el generador redirige las instrucciones a una lista diferida (instruccionesDiferidas) y las inserta al final del cuerpo, justo antes del salto de vuelta a la comparación.

break: se mantiene una pila con la etiqueta de fin de cada ciclo abierto. Un break emite goto a la etiqueta del tope, que siempre es la del ciclo más interno, lo que hace que funcione bien con ciclos anidados.

Ejemplo: para el while del caso válido se genera:

```
principal_while_0_inicio:
t7 = a > 0
if t7 goto principal_while_0_bloque
goto principal_while_0_fin
principal_while_0_bloque:
t8 = a - 1
a = t8
t9 = a == 5
ifFalse t9 goto principal_if_1_siguiente
principal_if_1_bloque:
goto principal_while_0_fin
principal_if_1_siguiente:
goto principal_while_0_inicio
principal_while_0_fin:
```

Recuperación de errores: modo pánico

Para los errores sintácticos se mantuvo la recuperación en modo pánico con producciones como SENTENCIA ::= error FIN_EXPR y ELEMENTO_PROGRAMA ::= error FIN_EXPR: si

el parser no entiende una sentencia, descarta tokens hasta encontrar el siguiente » y sigue desde ahí. Los errores léxicos se siguen acumulando en una cola dentro del lexer y los semánticos en su propia lista, así que ninguno de los tres tipos detiene la ejecución. El resultado es que una sola corrida puede reportar todos los errores del archivo, cada uno en su archivo y con su línea.

Librerías usadas

- **JFlex** (jflex-full-1.9.1.jar): genera Lexer.java a partir de Lexer.flex.
- **CUP** (java-cup-11b-20151001.jar): genera Parser.java y sym.java a partir de Parser.cup.
- **CUP runtime** (java-cup-runtime-11b-20151001.jar): clases Symbol y Scanner que el parser generado necesita para correr.
- **java.io** (FileReader, FileWriter, IOException): leer el archivo fuente y escribir tokens.txt, la tabla de símbolos y los archivos de errores.
- **java.nio.file** (Files, Path): escribir el archivo codigo_intermedio.txt con todas las instrucciones.
- **java.nio.charset.StandardCharsets**: forzar UTF-8 al leer y escribir (el lenguaje usa símbolos Unicode).
- **java.util.ArrayDeque / Deque**: cola de errores léxicos, pila de ámbitos y pilas de contextos de if, for, while y etiquetas de fin de ciclo.
- **java.util.ArrayList / List**: listas de errores, símbolos, instrucciones, argumentos y tipos de parámetros.
- **java.util.HashSet / Set**: detectar nombres duplicados en cada ámbito.
- **java.util.HashMap / Map**: contadores para numerar los ámbitos de cada estructura (if 1, while 2, …).
- **java.util.Collections**: devolver las listas de la tabla de símbolos como vistas inmutables.

Análisis de resultados: objetivos alcanzados, objetivos no alcanzados, y razones por las cuales no se alcanzaron los objetivos (en caso de haberlos).

Objetivos alcanzados

- Se preservaron los alcances del Proyecto I: el lexer, la gramática y la tabla de símbolos por ámbitos siguen funcionando igual.
- Detección de identificadores no declarados o fuera de su ámbito, y de identificadores repetidos en un mismo ámbito.
- Validación de tipos en operaciones aritméticas, relacionales y lógicas, en asignaciones, en declaraciones con valor inicial y en las condiciones de if, elif, while y for.

- Validación de funciones: existencia, cantidad y tipo de los argumentos, tipo del valor retornado, return obligatorio en funciones con tipo y que principal no retorne valores.
- Validación de arreglos: tamaño mayor que cero, cantidad y tipo de los elementos iniciales, índice entero, índice constante dentro de los límites y tipo del valor asignado a un elemento.
- Validación de read y write según el tipo de la variable, y de break solo dentro de un ciclo.
- Manejo de errores léxicos, sintácticos (modo pánico) y semánticos sin detener el análisis; cada tipo se reporta en su archivo con la línea donde ocurrió.
- El programa indica correctamente si el archivo pertenece o no a la gramática tomando en cuenta la gramática, la sintaxis y la semántica: un archivo con errores semánticos se rechaza aunque su sintaxis sea correcta.
- Generación de código intermedio de tres direcciones para declaraciones, asignaciones, expresiones, arreglos, funciones, parámetros, llamadas, return, if/elif/else, while, for, break (incluyendo ciclos anidados), read y write.

Objetivos no alcanzados

- Puntos extra: no se implementó el manejo de clases ni técnicas de recuperación adicionales.

Razones

- Se priorizó completar y probar los requerimientos funcionales obligatorios dentro del tiempo disponible, antes que los puntos opcionales.

Justificación de toma de decisiones: forma en que se realizó la validación semántica y sus estructuras.

ResultadoExpresion: llevar el tipo y el lugar juntos

Al principio las expresiones solo devolvían un String con su tipo, lo cual alcanzaba para validar, pero no para generar código. Cuando empezamos con el código intermedio necesitábamos saber, además del tipo, dónde quedó guardado el valor (un temporal, una variable o un literal). Por eso creamos la clase ResultadoExpresion con dos campos, tipo y lugar, y adaptamos todas las reglas de expresiones para que devuelvan ese objeto. Así la misma reducción valida y genera: con el tipo se decide si la operación es válida y qué tipo resulta, y con el lugar se emite la instrucción t = izquierdo op derecho.

Propagación de errores con null

Si una subexpresión tiene un error semántico, su ResultadoExpresion es null. Decidimos esto para que un solo error no genere una lista de errores falsos más arriba: todas las validaciones

y el generador revisan primero si les llegó null y, si es así, no reportan nada ni emiten código para esa expresión. El error se reporta una sola vez, en el lugar donde realmente ocurrió.

Reglas de tipos

Como el lenguaje es de tipado fuerte, no hay conversiones implícitas en las asignaciones: el tipo de la expresión tiene que ser exactamente el tipo declarado. Para los operadores definimos esta tabla:

Operador	Operandos permitidos	Tipo resultante
+ - *	int o float	float si alguno es float; si no, int
/	int o float	float
// mod pot	solo int	int
< <= > >=	int o float	bool
== !=	numérico con numérico, bool con bool, char con char	bool
λ (and) θ (or)	solo bool	bool
Σ (not)	solo bool	bool
++ --	variable int o float	el de la variable

La división normal siempre da float porque su resultado puede tener decimales; la división entera, el módulo y la potencia quedaron solo para enteros porque así se definieron en el lenguaje. Además, las condiciones de if, elif, while y for tienen que ser de tipo bool: no se acepta un entero como condición.

Funciones: firma, argumentos y return

Cuando se declara una función se registra en el ámbito global con su tipo de retorno y, al terminar de leer los parámetros, se guarda su firma (lista de tipos). La función se registra antes de abrir su cuerpo, por lo que también se puede llamar a sí misma. En cada llamada se busca la función en el ámbito global y se compara la cantidad y el tipo de cada argumento contra la firma, reportando exactamente qué argumento está mal.

Para el return se usa una bandera encontroReturn que se reinicia al entrar a cada función. Cada return compara el tipo de su expresión con el tipo de la función actual (que se obtiene recorriendo la pila de ámbitos), y al cerrar la función, si la bandera sigue en falso, se reporta que falta el return. En principal, que es void, cualquier return con valor es un error.

Arreglos

La tabla de símbolos guarda el tamaño de cada arreglo y su tipo como tipo[]. Al declararlo se valida que el tamaño sea positivo y que la lista de inicialización tenga exactamente esa cantidad de elementos, todos del tipo del arreglo. En cada acceso se valida que el identificador realmente sea un arreglo, que el índice sea int y, si el índice es un literal, que esté dentro de los límites. Los índices que son variables o expresiones no se pueden comprobar en tiempo de compilación porque su valor se conoce hasta ejecutar, así que para esos solo se valida el tipo. El tipo de un elemento se obtiene quitándole el [] al tipo del arreglo.

break, read y write

Para el break no hizo falta una estructura nueva: como cada while y for abre un ámbito con su nombre, basta con revisar si en la pila de ámbitos hay alguno de ciclo. read solo acepta variables o parámetros de tipo int, float o char, y write acepta literales o variables de tipo int, float, bool o string, de acuerdo con lo definido para el lenguaje.

Por qué todo en la misma pasada

Hacer el análisis semántico y la generación de código dentro de las acciones del parser nos permitió reutilizar la tabla de símbolos exactamente en el estado en que está en ese punto del programa, sin tener que reconstruirla en una segunda pasada. También hace que los errores se detecten justo en la línea donde ocurren. La desventaja es que el .cup quedó más grande, por eso sacamos toda la lógica de validación a métodos con nombre en el bloque parser code y la de generación a la clase GeneradorCodigoIntermedio, para que las acciones de las reglas queden cortas y fáciles de leer.

Bitácora

Registro de commits en el repositorio: https://github.com/Tayler1203/Proyecto2Compiladores